

前端转 Agent 学习指南

基于 autobots & agplateform 项目实战 面向:前端工程师转 AI Agent 方向 你的优势:autobots 前端是你做的,SSE / 语音 / Page Agent SDK 你已经会一半

目录

1. 为什么前端转 Agent 是当下最好的窗口
2. Agent 是什么 — 用前端能懂的话讲
3. 你已经会的:从 autobots 前端切入
4. Agent 的四大支柱:LLM / Tool / Memory / RAG
5. SSE 流式协议 — 你的起点
6. 浏览器内 Agent — Page Agent SDK
7. 语音 Agent — ASR / TTS / VAD 三件套
8. Python Agent Runtime — 从 FastAPI 学起
9. Rust Runtime — 新一代高性能 Agent
10. MCP 协议 — Agent 的"USB 接口"
11. RAG 检索增强 — 让 Agent 有知识
12. 多 Agent 协作 — A2A 与 Swarm
13. 编排层 — Go Orchestrator
14. 90 天学习路径
15. 你现在就能接手的 Codex 全栈任务
16. 关键设计模式速查表
17. 名词速查

1. 为什么前端转 Agent 是当下最好的窗口

1.1 行业窗口

- **Agent 是 2025–2026 的主战场**:OpenAI、Anthropic、阿里、字节都在押注 Agent
- **前端在 Agent 链路里价值极高**:流式聊天、语音、浏览器内 Agent、可视化编排,全是前端活
- **Codex / Cursor / Claude Code 让前端能全栈**:你现在用 Codex 一把梭前后端,说明工具链已经成熟
- **Agent 前端 ≠ 传统前端**:要懂 SSE、音频流、工具调用 UI、状态机,门槛比普通中后台高,所以溢价高

1.2 你的独特优势(autobots 前端经验)

你做过这些东西,这些都是 Agent 前端的核心能力:

你做过的	在 Agent 领域叫什么	重要程度
SSE 流式聊天 UI	Streaming Chat	★★★★★
语音通话前端	Realtime Voice Agent	★★★★★
AudioWorklet PCM 采集	Browser Audio Pipeline	★★★★★
Page Agent SDK	In-page Agent	★★★★★
React 状态管理	Agent State Machine	★★★★★

结论:你不是"转行",而是"把前端能力延伸到 Agent 链路"。后端部分用 Codex 帮你写,你只需要"看懂 + 能改 + 能调"。

2. Agent 是什么 — 用前端能懂的话讲

2.1 一句话定义

Agent = LLM(大脑) + Tool(手) + Memory(记忆) + Loop(循环)

普通聊天机器人是"一问一答",Agent 是"自己想、自己做、做完反馈、再想"的循环系统。

2.2 用前端类比

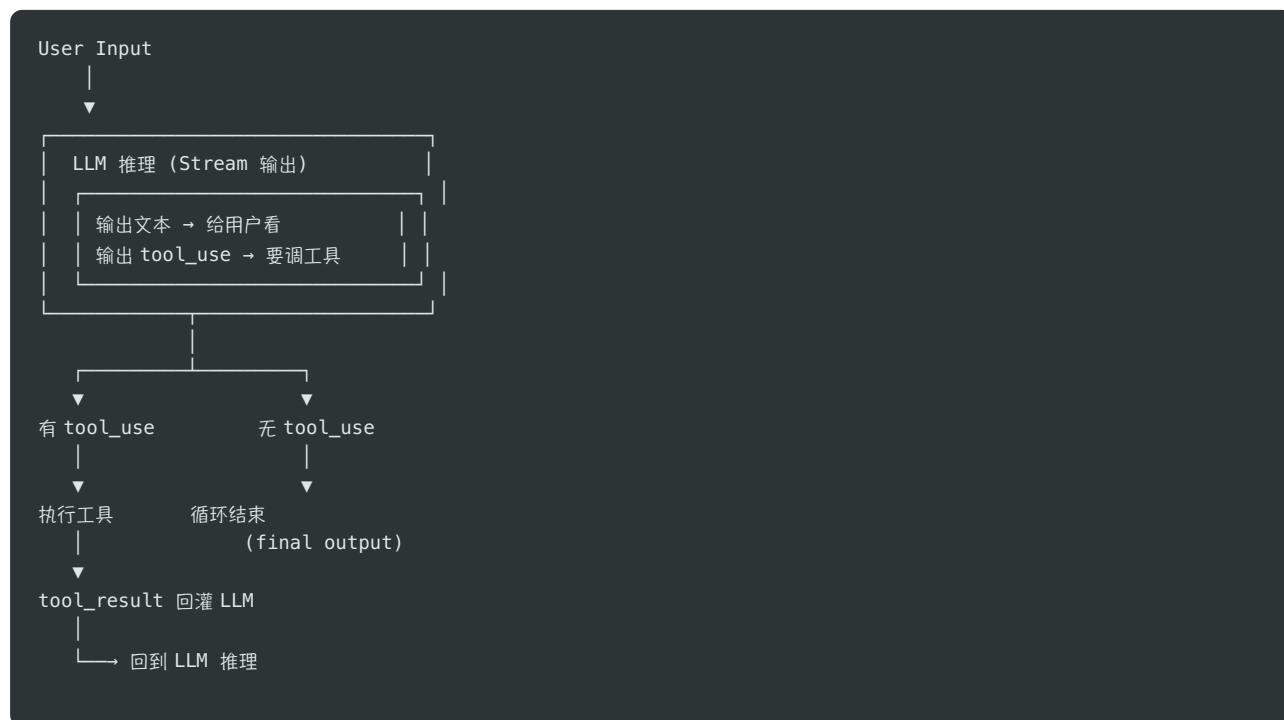
把 Agent 想象成一个"会自己写代码的实习生":

```
你(用户) → 提需求
↓
Agent(LLM) → 想思路(thinking)
↓
Agent 调工具(tool_call) → 比如读文件、查数据库、调 API
↓
工具返回结果(tool_result)
↓
Agent 看结果 → 还没做完?继续调工具
↓
做完了 → 输出最终结果(output)
```

这就像你写 React 时:

- `useState` = Agent 的 Memory
- `useEffect` 调 API = Agent 的 Tool call
- 组件循环渲染 = Agent 的 Loop

2.3 Agent 的核心循环(必背)



这个循环在 agplatform 的 Rust runtime 里叫 `loop_core`, 在 Python runtime 里叫 `AgentRunner`, 是所有 Agent 框架的核心。

3. 你已经会的:从 autobots 前端切入

3.1 autobots 前端技术栈回顾

```
autobots-frontend (Vue 3 + TS)
├─ app-manager    # 主管理端
├─ app-admin      # 企业管理
└─ app            # C 端聊天
```

agplatform frontend (React 18 + TS):

- `useSSEChat.ts` — SSE 流式聊天
- `useVoiceSession.ts` — 完整语音会话
- `useAsrSession.ts` — 语音识别
- `useTtsPlayback.ts` — 语音合成
- `pageAgent.tsx` — 浏览器内 Agent
- `silero_vad_v5.onnx` — VAD 模型
- `pcm-capture.worklet.js` — 音频采集

3.2 你写过的 SSE 聊天,在 Agent 链路里是什么

你前端写的 `useSSEChat`, 本质上是在消费 Agent 的流式输出。Agent 后端会吐出这些事件:

```
event: connected      → 连接建立
event: thinking        → Agent 在思考(显示 loading)
event: tool_call       → Agent 调工具(显示工具卡片)
event: output          → 流式文本(逐字显示)
event: completed       → 完成
event: error           → 出错
```

你前端的 rAF 批处理、错误区分、重试,这些全都是 Agent 前端的核心技能。后端只是把这些事件用 SSE 推给你。

3.3 你能立即做的事

1. 把 autobots 的 SSE 聊天抽成通用组件,以后任何 Agent 项目都能复用
2. 学习 Agent 事件协议(看 agplatform `runtime/agentic_runtime/api/sse.py`),理解前端为什么这么写
3. 用 Codex 给你的 SSE 组件加工具调用 UI(显示 Agent 调了哪些工具、参数、返回值)

4. Agent 的四大支柱:LLM / Tool / Memory / RAG

4.1 LLM(大脑)

前端类比:LLM 就是你调的"AI 接口",但 Agent 场景比普通聊天复杂。

项目里用到的 LLM:

Provider	用途	文件
DeepSeek	autobots 主力	<code>autobots-ai/ai/llm_loader.py</code>
通义千问(qwen-flash)	云上快模型	同上
qwen2.5-72b	自建推理集群	<code>pre-llm-inference-api.brapp.com</code>
Anthropic Claude	agplatform Rust runtime	<code>rust/crates/ap-runtime/src/model/</code>
OpenRouter	多模型路由	Nacos shared-llm.yaml

OpenAI 兼容协议(必懂): 所有 LLM 都用同一套 API: `POST /v1/chat/completions` ,换 `base-url` 和 `api-key` 就能切模型。

```
# 伪代码,所有兼容 OpenAI 的模型都这么调
client = ChatOpenAI(
    model="qwen-flash",
    openai_api_base="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key="...",
)
```

学习要点:

- `temperature=0` → 确定性输出(业务场景)
- `temperature=0.7` → 创意输出(对话场景)
- `streaming=True` → 流式输出(前端 SSE 消费)

4.2 Tool(手)

Agent 调工具用的是 **OpenAI function-calling** 协议。前端要理解这个,因为它决定了你怎么渲染工具 UI。

工具定义(JSON Schema 格式):

```
{
  "type": "function",
  "function": {
    "name": "create_agent",
    "description": "创建一个子 Agent",
    "parameters": {
      "type": "object",
      "properties": {
        "role": {"type": "string", "description": "角色名"},
        "guidance": {"type": "string", "description": "系统指令"}
      },
      "required": ["role"]
    }
  }
}
```

LLM 会输出:

```
{"name": "create_agent", "arguments": {"role": "researcher"}}
```

你前端就可以渲染成一张"创建 Agent"的卡片,显示参数。

agplatform 的 6 个内置工具(Claude Code 同款):

- **Bash** — 执行命令
- **FileRead** / **Write** / **Edit** — 文件操作
- **Glob** / **Grep** — 搜索

Swarm 多 Agent 协作工具:

- **self** / **list_agents** / **create** / **send** / **create_group** 等

4.3 Memory(记忆)

前端类比:Memory 就是 Agent 的 **useState** + **localStorage**。

项目里的三层记忆:

- **会话记忆(working)**:当前对话上下文 → 类比 React state
- **长期记忆(long-term)**:跨会话记住用户偏好 → 类比 localStorage
- **无损重述(lossless_restatement)**:原始记忆存储

autobots 用 **ChatMemoryManager** + **SimpleMemoryManager**。agplatform 用 LanceDB 做向量记忆检索 (**memory.search**)。

4.4 RAG(知识)

前端类比:RAG 就是"给 Agent 配一个搜索引擎"。

用户问:"公司报销流程是什么?"

↓

1. 把问题转成向量(embedding)
2. 在知识库向量索引里找最相关的文档片段
3. 把找到的片段塞进 LLM 的 prompt
4. LLM 基于这些片段回答

autobots 的 RAG 用 **Milvus 混合检索**(dense + sparse),agplatform 用 **LanceDB**。

5. SSE 流式协议 — 你的起点

5.1 为什么 Agent 必须用 SSE

LLM 推理慢(几秒到几十秒),如果用普通 HTTP,用户要等很久才看到结果。SSE 让你"边生成边显示",体验质的飞跃。

5.2 SSE 事件协议(项目实战)

agplatform 的 `api/sse.py` 定义了 Agent 执行的事件类型:

```
# 事件优先级(队列满时丢低优先级)
PRIORITY = {
    "connected": 1,      # 连接(最低)
    "thinking": 2,       # 思考中
    "tool_call": 3,       # 工具调用
    "output": 4,          # 输出文本(最高)
    "completed": 5,       # 完成
    "error": 5,           # 错误
}
```

SSE 文本格式:

```
event: thinking
data: {"agent_id": "xxx"}

event: tool_call
data: {"tool": "FileRead", "args": {"path": "/tmp/a.py"}}

event: output
data: {"delta": "你好"}

event: output
data: {"delta": ", 世界"}

event: completed
data: {}
```

5.3 前端消费 SSE(你已经会)

```
// 你在 autobots 写过的伪代码
const eventSource = new EventSource('/api/chat/stream');
eventSource.addEventListener('thinking', (e) => setThinking(true));
eventSource.addEventListener('tool_call', (e) => {
  const tool = JSON.parse(e.data);
  setTools(prev => [...prev, tool]);
});
eventSource.addEventListener('output', (e) => {
  const { delta } = JSON.parse(e.data);
  setMessage(prev => prev + delta); // 逐字拼接
});
```

5.4 进阶:为什么用 rAF 批处理

LLM 一秒能吐几十个 chunk,直接 `setState` 会卡。你用 `requestAnimationFrame` 批量 flush,这是 Agent 前端的标准优化手段。

5.5 学习任务

- [] 读懂 `agplatform/runtime/agentic_runtime/api/sse.py` 的队列实现
- [] 把 autobots 的 SSE 组件抽象成通用 `<AgentChat />` 组件
- [] 加上工具调用卡片 UI(显示工具名、参数、结果、耗时)

6. 浏览器内 Agent — Page Agent SDK

6.1 什么是 Page Agent

agplatform 有个 `page-agent-sdk.js`,让 Agent 能直接操作网页 DOM。这是浏览器端 Agent 的关键能力(类似 Anthropic 的 Computer Use,但更轻量)。

6.2 工作流程



6.3 前端价值

这是你最该深挖的方向。浏览器内 Agent 是 2026 年的爆款方向(Browser Use、Anthropic Computer Use、OpenAI Operator 都是)。

学习任务:

- [] 读懂 `frontend/src/utils/pageAgent.tsx`
- [] 理解 HMAC 握手流程(`rust/crates/ap-runtime/src/page_agent/`)
- [] 用 Codex 写一个"自动填表单"的 demo

7. 语音 Agent — ASR / TTS / VAD 三件套

7.1 你做过的,在 Agent 领域的术语

你做的	术语	作用
<code>useAsrSession.ts</code>	ASR (Automatic Speech Recognition)	语音转文字
<code>useTtsPlayback.ts</code>	TTS (Text-to-Speech)	文字转语音
<code>pcm-capture.worklet.js</code>	AudioWorklet PCM Capture	浏览器音频采集
<code>silero_vad_v5.onnx</code>	VAD (Voice Activity Detection)	检测说话开始/结束

7.2 完整语音 Agent 闭环

```
用户说话 → 麦克风采集 PCM
↓
VAD 检测到说话 → 开始录音
VAD 检测到停顿 → 结束录音
↓
ASR 转文字 → 发给 LLM
↓
LLM 流式回复 → TTS 流式播放
↓
用户听到回答 → 可以打断(barge-in)
```

7.3 Silero VAD(值得学)

项目用 `silero_vad_v5.onnx`, 这是 PyTorch 训练的 VAD 模型, 导出成 ONNX 在浏览器跑(ONNX Runtime Web)。

为什么不用 Web Speech API:

- Web Speech API 浏览器兼容性差
- Silero VAD 是真模型, 准确率高, 能在本地跑
- 配合自建 ASR 服务, 延迟可控

7.4 学习任务

- [] 理解 AudioWorklet(比 ScriptProcessorNode 现代)
- [] 跑通 Silero VAD ONNX 在浏览器的推理
- [] 实现一个"打断"逻辑(用户说话时停止 TTS 播放)

8. Python Agent Runtime — 从 FastAPI 学起

8.1 为什么前端要懂后端

你现在用 Codex 一把梭, 但不懂后端就调不好 Codex。你要能:

- 看懂 Codex 生成的代码对不对
- 出 bug 时知道是前端还是后端问题
- 改小 bug 不用每次都让 Codex 重写

8.2 Python Runtime 架构(agplatform/runtime/)

```
agentic_runtime/
├── main.py           # FastAPI 入口
├── api/
│   ├── deps.py      # 依赖注入(从 header 取 tenant_id)
│   └── sse.py        # SSE 流式响应
├── model/
│   ├── base.py       # LLM 适配器抽象
│   └── types.py      # ChatMessage / ChatResponse / StreamChunk
├── mcp/
│   ├── manager.py    # MCP 连接管理
│   └── types.py      # MCP 协议类型
├── skill/sdk.py      # Skill SDK
├── swarm/
│   ├── bus.py        # 事件总线
│   ├── store.py      # 持久化
│   └── tools.py       # 协作工具
└── task/store.py     # 异步任务
```

8.3 LLM 适配器抽象(设计模式:策略 + 模板方法)

```
class BaseModelAdapter(ABC):
    @abstractmethod
    async def chat_async(self, messages, tools=None) -> ChatResponse: ...
    @abstractmethod
    async def stream(self, messages, tools=None) -> AsyncIterator[StreamChunk]: ...

    def format_tools(self, tools):
        return [tool.to_openai_format() for tool in tools] # 默认 OpenAI 格式
```

前端类比:这就是你写 React 时的 `interface Props`,定义了"所有 LLM 都得有这些方法"。子类(OpenAI/Anthropic/DeepSeek)各自实现。

8.4 依赖注入(FastAPI 风格)

```
# api/deps.py
def get_tenant_id(x_internal_tenant_id: str = Header(...)) -> str:
    if not x_internal_tenant_id:
        raise HTTPException(401)
    return x_internal_tenant_id
```

前端类比:类似 React Context,从全局拿 `tenant_id`。FastAPI 用 `Header()` 自动从请求头提取。

8.5 学习任务

- [] 读懂 `model/base.py` 的抽象设计
- [] 读懂 `api/sse.py` 的事件队列
- [] 用 Codex 生成一个最简 Agent:接 OpenAI,流式返回

9. Rust Runtime — 新一代高性能 Agent

9.1 为什么有 Rust Runtime

Python runtime 性能不够(并发低、内存占用大),agplateform 正在迁移到 Rust。Rust runtime 作为 `ap-gateway` 的库内嵌,不单独起进程。

9.2 Rust 你需要懂多少

不需要会写,但要能"读"。重点术语:

Rust 概念	前端类比
<code>struct</code>	TypeScript interface
<code>enum</code>	TypeScript union type
<code>trait</code>	TypeScript interface + 实现
<code>impl</code>	class 方法
<code>async/await</code>	一样的
<code>Result<T, E></code>	try/catch 的类型化版本
<code>Option<T></code>	<code>T null</code>
<code>Box<T></code>	堆分配(类似 <code>new</code>)
<code>Arc<T></code>	引用计数(共享所有权)

9.3 核心 crate:ap-runtime

```
rust/crates/ap-runtime/src/
├─ agent/loop_core.rs      # Agent 主循环(tool_use 多轮)
├─ model/
│   └─ openai_compat.rs    # OpenAI 兼容(覆盖 5 个 provider)
├─ tool/builtin/
│   ├── bash.rs / file_read.rs / write.rs / edit.rs
│   └── glob.rs / grep.rs  # 6 内置工具
├─ mcp/client.rs           # MCP 客户端
├─ skill/                  # SKILL.md 解析 + SkillRunner
├─ task/                   # Redis 异步任务
├─ page_agent/             # HMAC 会话令牌
└─ http/                  # /v1/agents/* 路由
```

9.4 OpenAI 兼容适配器(一个适配 5 个 provider)

`OpenAiCompat` 实现 `ModelProvider` trait,一个适配器覆盖:

- OpenAI
- DeepSeek
- DashScope(阿里通义)
- Ollama(本地模型)

- OpenRouter

学习要点:这就是"适配器模式",换个 `base_url` 就能用不同模型。

9.5 Anthropic 适配器(手写 SSE)

Rust 生态没有成熟的 Anthropic SDK,所以项目用 `eventsource-stream` 自己解析 SSE。这是工程能力的体现。

9.6 学习任务

- [] 能读懂 `loop_core.rs` 的主循环逻辑
- [] 理解 `trait ModelProvider` 的抽象
- [] 不需要会写,出 bug 能定位

10. MCP 协议 — Agent 的"USB 接口"

10.1 什么是 MCP

MCP (Model Context Protocol) 是 Anthropic 2024 年提出的开放协议,让 Agent 能标准化地调用外部工具/资源/Prompt。

前端类比:MCP 之于 Agent,就像 REST API 之于前端。有了标准协议,Agent 调工具不用每个都写适配。

10.2 MCP 三类能力

Tools → 可调用的函数(类似 OpenAI function-calling)
Resources → 可读取的数据(文件、数据库记录)
Prompts → 可复用的提示词模板

10.3 MCP 协议(JSON-RPC)

```
// 客户端 → 服务端
{"method": "initialize", "params": {...}}

// 列出工具
{"method": "tools/list"}

// 调用工具
{"method": "tools/call", "params": {"name": "get_weather", "arguments": {"city": "北京"}}
```

10.4 传输方式

传输	场景
STDIO	本地进程(比如 Claude Desktop 调本地 MCP)
HTTP	普通远程调用
SSE	旧版流式(服务端推送)
Streamable HTTP	新版,推荐(HTTP + 可选 SSE)

agplatform 的 Rust runtime 实现了 Streamable HTTP 客户端。

10.5 学习任务

- [] 读懂 `runtime/agentlic_runtime/mcp/manager.py`
- [] 读懂 `rust/crates/ap-runtime/src/mcp/client.rs`
- [] 用 Codex 写一个最简 MCP server(提供 1 个工具)

11. RAG 检索增强 — 让 Agent 有知识

11.1 RAG 流程

文档入库：

PDF/Word → 切片(chunk) → embedding → 存向量库

查询：

用户问题 → embedding → 向量库找相似 → 拼 prompt → LLM 答

11.2 混合检索(autobots 的核心)

autobots 用 Milvus 2.6 Hybrid Search,三路融合:

```
class HybridMilvusRetriever:
    alpha: float = 0.6 # 稠密权重
    beta: float = 0.4 # 稀疏权重
    rerank_strategy: str = "rrf" # 或 "weighted_sum"

    def _get_relevant_documents(self, query, expr=None):
        q_dense = self.embeddings.embed_query(query) # 语义向量
        q_sparse = self.sparse_encoder(query) # 关键词向量
        # Milvus 同时做 dense ANN + sparse + expr 过滤
        res = self.collection.search(
            data=[q_dense],
            sparse_vectors={"sparse_vector": [q_sparse]},
            rerank={"strategy": "rrf", "weight": [alpha, beta]},
            expr=final_expr, # 元数据过滤(如 tenant_id)
        )
```

11.3 两路检索原理

类型	擅长	例子
Dense(稠密向量)	语义匹配	"开心" ≈ "高兴"
Sparse(稀疏向量)	关键词匹配	"百融" 专有名词

RRF (Reciprocal Rank Fusion) 融合公式:

```
score =  $\sum 1/(k + \text{rank}_i)$     # k 通常取 60
```

不需要归一化分数,简单有效。

11.4 两阶段检索(粗排 + 精排)

```
Query → HybridMilvusRetriever (粗排, 取 top 50)
      → Reranker (精排, 取 top 5)
          ├── custom: 外部 API, 失败回退本地
          ├── rankllm: 用 LLM 打分
          └── llm_extractor: LLM 抽关键句
```

11.5 LanceDB vs Milvus

agplateform 用 LanceDB 取代 Milvus:

- 嵌入式:无需独立服务,像 SQLite
- 多视图索引:向量 + FTS + metadata 一起查
- 适合 Agent 场景:轻量、快

11.6 学习任务

- [] 读懂 `autobots-ai/ai/retrievers/HybridMilvusRetriever.py`
- [] 理解 RRF 融合
- [] 用 Codex 跑一个最简 RAG(3 个文档切片 + 检索)

12. 多 Agent 协作 — A2A 与 Swarm

12.1 为什么要多 Agent

单个 Agent 能力有限,多个 Agent 分工协作:

- Researcher Agent 负责查资料
- Coder Agent 负责写代码
- Reviewer Agent 负责审查
- 它们之间要通信

12.2 A2A (Agent-to-Agent) — Go 编排层

agplatform 的 `orchestrator/` (Go)负责 Agent 间通信。

Agent Card 概念:每个 Agent 有一张"名片",描述能力、endpoint,其他 Agent 通过名片找到它。

Resolver 装饰器链(设计模式:装饰器):

```
Nacos Resolver (主)
  ↓ 包装
Hybrid Resolver (回退到 Java 后端)
  ↓ 包装
Cached Resolver (Redis 缓存)
```

三种消息路由:

- `direct` — HTTP 直连(简单)
- `rocketmq` — 消息队列(异步、解耦)
- `hybrid` — 混合(默认 direct,RocketMQ 不可用降级)

12.3 Swarm — Python 多 Agent 协作

agplatform 的 `runtime/agentic_runtime/swarm/` 实现"群聊式"多 Agent。

双层事件总线(精妙设计):

```
class AgentEventBus:
    """进程内 per-agent(asyncio.Queue)
    事件:wakeup / unread / stream / done / error
    队列满丢旧事件(背压)"""

class WorkspaceUIBus:
    """跨进程 per-workspace(Redis pub/sub)
    事件推送到前端 SSE
    fire-and-forget,带任务跟踪
    关闭时 asyncio.wait(timeout=5) 优雅退出"""
```

内置协作工具(OpenAI function-calling 格式):

- `self` — 获取自己的身份
- `list_agents` — 列出所有 Agent
- `create` — 创建子 Agent(自动建 P2P 群)
- `send` / `send_group_message` — 点对点/群消息
- `create_group` / `list_groups` / `get_group_messages`

12.4 学习任务

- [] 读懂 `orchestrator/internal/a2a/manager.go` 的 Resolver 链
- [] 读懂 `swarm/bus.py` 的双层事件总线
- [] 理解为什么进程内用 Queue、跨进程用 Redis pub/sub

13. 编排层 — Go Orchestrator

13.1 Orchestrator 职责

```
orchestrator/ (Go)
├─ a2a/          # Agent 间通信
├─ canary/       # 灰度发布
├─ sandbox/      # 沙箱隔离
├─ workflow/     # 工作流引擎
└─ cache/        # Redis 缓存
```

13.2 Canary 灰度发布

为 Agent runtime 做渐进上线:

- 百分比灰度:10% 流量到新版本
- 租户灰度:指定租户先用
- Sticky Session:同一会话粘到同一版本

13.3 Sandbox 沙箱

Agent 执行代码要隔离:

- `docker` — 容器隔离
- `gvisor` — Google gVisor(系统调用级,更安全)
- `none` — 不隔离(开发用)

13.4 学习任务(可选,优先级低)

- [] 了解 Go 的 goroutine、channel(类比 Promise.all)
- [] 读 `canary/types.go` 理解灰度策略
- [] 不需要会写 Go

14. 90 天学习路径(可执行手册)

每一项都包含:读什么 / 怎么练 / 验证标准 / Codex 怎么用。

W1:读懂 autobots SSE 聊天实现

为什么先学这个:你做过 autobots 前端,从你写的代码切入,建立"我懂一半"的信心。

读什么(按顺序):

1. `autobots-frontend/app-manager/src/api/chat.ts` — 你写过的 SSE 调用
2. `autobots-frontend/app-manager/src/api/chatHistory.ts` — 历史消息
3. agplatform `frontend/src/hooks/useSSEChat.ts` — 对照实现(React 版)
4. agplatform `runtime/agentlic_runtime/api/sse.py` — 后端怎么发事件

怎么练(3 步):

1. 用纸笔把 autobots SSE 的事件流画出来: 连接 → thinking → output → completed
2. 对比 agplateform 的 `useSSEChat.ts`, 找出两者处理方式的差异(比如重试、错误区分)
3. 读 `api/sse.py`, 理解前端收到的每个事件后端是怎么发的

验证标准:

- 能口述"SSE 从用户发消息到看到回答,中间发生了什么"
- 能说出 `thinking` 和 `output` 事件的区别
- 能解释为什么用 SSE 不用 WebSocket(Agent 是单向流式输出,SSE 更简单)

Codex 怎么用:

- 让 Codex 解释 `api/sse.py` 里"事件优先级队列"那段代码
- 让 Codex 给你写一段"为什么 SSE 队列满了要丢事件"的注释

时间投入:7-10 小时(每天 1 小时)

W2:抽象通用 `<AgentChat />` 组件

为什么学这个:把已有能力沉淀成可复用组件,是前端转 Agent 的第一步产出。

读什么:

1. 复读 W1 的文件,这次关注"可抽象点"
2. 看 agplateform `frontend/src/hooks/useSSEChat.ts` 的公开 API 设计(参数、返回值)
3. 参考 shadcn/ui 或 Ant Design 的组件 API 设计风格

怎么练(4 步):

1. 用纸设计 `<AgentChat />` 的 Props: `<AgentChat agentId="xxx" sessionId="xxx" streamUrl="/v1/agents/xxx/execute" onToolCall={({tool}) => ...} onMessage={({msg}) => ...} />`
2. 让 Codex 基于 autobots 现有代码生成骨架
3. 你手动调整 API,确保支持:流式文本、工具调用、思考状态、错误重试
4. 写一个 Storybook 或 demo 页面展示三种状态/loading / streaming / error)

验证标准:

- 组件能接任意 OpenAI 兼容的 Agent 后端
- 在 autobots 里替换原有聊天 UI,功能不退化
- 组件代码 < 300 行,API 清晰

Codex 怎么用:

- "参考 autobots-frontend 的 chat.ts,帮我生成一个 React AgentChat 组件,Props 是 ..."
- 让 Codex 写测试用例(mock SSE 事件)

时间投入:10-14 小时

W3:学 OpenAI function-calling 协议

为什么学这个:工具调用是 Agent 的核心,不懂协议就看不懂 Agent 在干什么。

读什么:

1. agplateform `runtime/agent_runtime/swarm/tools.py` — 真实工具定义

1. agplatform `runtime/agentic_runtime/model/base.py` 的 `format_tools` 方法
3. OpenAI 官方文档 function-calling(用 WebSearch 找最新版)
4. Anthropic tool_use 文档(对照看差异)

怎么练(4 步):

1. 抄一个 `swarm/tools.py` 里的工具定义(比如 `create_agent`),手写一遍
2. 用 curl 调 OpenAI API,看 LLM 返回的 tool_call 是什么格式: `bash curl https://api.openai.com/v1/chat/completions \ -H "Authorization: Bearer $KEY" \ -d '{"model":"gpt-4o","messages":[...],"tools":[...]}'`
3. 让 Codex 写一个 mock LLM,返回固定的 tool_call,你前端渲染
4. 实现工具卡片 UI:显示工具名、参数(JSON)、返回值、耗时、状态(pending/success/error)

验证标准:

- 能手写一个工具定义(不查文档)
- 能解释 tool_call 和 tool_result 的区别
- 工具卡片能正确渲染 3 种状态

Codex 怎么用:

- "帮我写一个 mock server,返回固定的 tool_call 事件"
- "这个工具定义的 JSON Schema 帮我检查对不对"

时间投入:10 小时

W4:学 Page Agent SDK

为什么学这个:浏览器内 Agent 是 2026 最火方向,你做过前端,这是你的护城河。

读什么:

1. agplatform `frontend/src/utils/pageAgent.tsx`
2. agplatform `frontend/public/page-agent-sdk.js` (可能是压缩版,先扫一遍)
3. agplatform `frontend/public/page-agent-test.html` (测试页)
4. agplatform `rust/crates/ap-runtime/src/page_agent/` (HMAC 握手,只读逻辑)

怎么练(4 步):

1. 跑通 `page-agent-test.html`,看 Agent 能操作页面
2. 抓包看握手流程:浏览器 → 后端,拿到 token 的请求长什么样
3. 让 Codex 写一个最简 demo:页面有个表单,Agent 自动填入数据并提交
4. 加一个"Agent 操作可视化"效果:Agent 点击的元素高亮闪烁

验证标准:

- 能说清楚 HMAC 握手为什么安全(防 token 被偷)
- demo 跑通:Agent 能读 DOM、点击、填表
- 能解释 Page Agent 和 Browser Use(开源项目)的异同

Codex 怎么用:

- "帮我在 `page-agent-test.html` 基础上加一个自动填表单的功能"
- "HMAC 握手那段代码帮我画个时序图"

时间投入:12-15 小时

W5:学 Python FastAPI + 异步

为什么学这个:你要能改后端,Python 是 agplatform runtime 的主语言。

读什么:

1. agplatform `runtime/agentc_runtime/main.py` — FastAPI 应用入口
2. agplatform `runtime/agentc_runtime/cli.py` — 启动流程
3. agplatform `runtime/agentc_runtime/api/deps.py` — 依赖注入
4. FastAPI 官方教程(中文)的"异步"和"依赖注入"两节

怎么练(4 步):

1. 在本地起一个最简 FastAPI: `python from fastapi import FastAPI app = FastAPI()`

`@app.get("/hello") async def hello(): return {"msg": "hi"} ```

1. 加一个 SSE 流式接口:

```
python from fastapi.responses import StreamingResponse @app.get("/stream") async def stream(): async def gen(): for i in range(5): yield f"data: chunk {i}\n\n" return StreamingResponse(gen(), media_type="text/event-stream")
```

2. 接 OpenAI,实现"用户提问 → LLM 流式回答"的最简 Agent
3. 用你 W2 写的 `<AgentChat />` 连接这个后端,端到端跑通

验证标准:

- 本地能 `uvicorn main:app` 起服务
- 能用 `async/await` 写异步函数
- 前端能消费到你后端的 SSE 流

Codex 怎么用:

- "帮我写一个 FastAPI 最简 SSE 接口,接 OpenAI streaming"
- 出错时让 Codex 解释报错原因(异步相关的坑很多)

时间投入:12-15 小时

W6:学 LLM 适配器抽象

为什么学这个:理解抽象层,以后加任何模型都不怕。

读什么:

1. agplatform `runtime/agentc_runtime/model/base.py` — 抽象基类
2. agplatform `runtime/agentc_runtime/model/types.py` — 数据类型
3. autobots `autobots-ai/ai/llm_loader.py` — 工厂函数风格对比
4. 复习 Python `abc` 模块和"策略模式"

怎么练(4 步):

1. 在纸上画 UML: `BaseModelAdapter` → 子类(OpenAI/DeepSeek/Anthropic)
2. 让 Codex 基于你 W5 的代码,重构出 `BaseModelAdapter`
3. 实现一个 `DeepSeekAdapter` 子类(参考 autobots 的 `loadmodel_deepseek`)
4. 加一个 `config.yaml`,运行时切换不同 provider

验证标准:

- 能说清楚"策略模式"在这个抽象里怎么用

- 切换 provider 不用改业务代码
- 能解释 `format_tools` 为什么是模板方法(子类可重写但不强制)

Codex 怎么用:

- "把我的 FastAPI 代码重构成 BaseModelAdapter 模式"
- "帮我写 DeepSeekAdapter,参考 autobots-ai/ai/llm_loader.py"

时间投入:10-12 小时

W7:学 MCP 协议

为什么学这个:MCP 是 Agent 调工具的标准协议,2026 年会爆发。

读什么:

1. agplatform `runtime/agent_runtime/mcp/types.py` — 协议类型
2. agplatform `runtime/agent_runtime/mcp/manager.py` — 连接管理
3. agplatform `rust/crates/ap-runtime/src/mcp/client.rs` — Rust 实现(对照)
4. MCP 官方规范(modelcontextprotocol.io)

怎么练(5 步):

1. 用纸画出 MCP 三次握手: `initialize` → `tools/list` → `tools/call`
2. 让 Codex 帮你写一个最简 MCP server(Python),提供 1 个工具 `get_time`
3. 用 curl 模拟客户端调你的 server
4. 把这个 MCP server 接到你的 Agent 上,Agent 能调 `get_time`
5. 加一个 Streamable HTTP 传输(不用 STDIO)

验证标准:

- 能说清楚 MCP 和 OpenAI function-calling 的关系(MCP 是更上层的协议)
- 你的 MCP server 能被标准 MCP client 调用
- 能解释为什么有 STDIO / HTTP / SSE / Streamable HTTP 四种传输

Codex 怎么用:

- "帮我写一个最简 MCP server,Python,提供 `get_time` 工具,用 Streamable HTTP"
- "我的 server 报错了,帮我看日志"

时间投入:12-15 小时

W8:学 RAG 基础

为什么学这个:让 Agent 有知识,是 80% 企业场景的需求。

读什么:

1. autobots `autobots-ai/ai/retrievers/HybridMilvusRetriever.py` — 混合检索
2. autobots `autobots-ai/ai/embedding_loader.py` — 向量化(注意是注释掉的代码,说明在迁移)
3. autobots `autobots-ai/ai/retrievers/RerankRetriever.py` — 两阶段检索
4. autobots `autobots-ai/ai/rerankers/custom_reranker.py` — 精排

怎么练(5 步):

1. 在纸上画 RAG 完整流程:入库(切片→embedding→存)+ 查询(query→embedding→检索→拼prompt)

2. 让 Codex 帮你用 LanceDB(轻量,不用起 Milvus)实现最简 RAG
3. 准备 3 个 txt 文档,入库
4. 实现"用户提问 → 检索 → LLM 回答"
5. 加一个 Reranker(用 LLM 打分),对比前后效果

验证标准:

- 能说清楚 dense 和 sparse 检索的区别
- 能解释 RRF 融合公式 $1/(k+rank)$
- 你的 RAG 能回答 3 个文档里的问题

Codex 怎么用:

- "帮我用 LanceDB + OpenAI embedding 写一个最简 RAG,Python"
- "帮我加一个 LLM Reranker,用 DeepSeek 给文档打分"

时间投入:15-18 小时

W9:学 Rust runtime(只读)

为什么学这个:agplatform 在迁 Rust,你要能看懂,出 bug 能定位。

读什么:

1. agplatform `rust/Cargo.toml` — workspace 结构
2. agplatform `rust/crates/ap-runtime/src/agent/loop_core.rs` — Agent 主循环
3. agplatform `rust/crates/ap-runtime/src/model/openai_compat.rs` — LLM 适配
4. agplatform `rust/crates/ap-runtime/src/tool/builtin/grep.rs` — 工具实现

怎么练(4 步):

1. 学 Rust 基础语法(只学 4 个概念): `struct` / `enum` / `trait` / `Result<T,E>`
2. 对照 `loop_core.rs`,画出 Agent 主循环流程图
3. 读 `openai_compat.rs`,理解一个适配器怎么覆盖 5 个 provider
4. 读 `grep.rs`,理解工具怎么定义和执行

验证标准:

- 能看懂 Rust 代码结构(不需要会写)
- 能指出"这是 trait 实现"、"这是 async 函数"、"这是 Result 返回"
- 能用 Rust 术语解释 loop_core 在做什么

Codex 怎么用:

- "这段 Rust 代码帮我逐行解释"
- "loop_core.rs 的 tool_use 循环画个流程图"

时间投入:12-15 小时

W10:学多 Agent 协作(Swarm)

为什么学这个:多 Agent 是 Agent 的进阶,也是企业场景的真实需求。

读什么:

1. agplatform `runtime/agentic_runtime/swarm/bus.py` — 双层事件总线

2. agplatform `runtime/agentic_runtime/swarm/store.py` — 持久化
3. agplatform `runtime/agentic_runtime/swarm/tools.py` — 协作工具
4. 复习 W3 学的 function-calling

怎么练(5 步):

1. 在纸上画双层事件总线: `AgentEventBus` (进程内 Queue)+ `WorkspaceUIBus` (Redis pub/sub)
2. 让 Codex 跑一个最简 Swarm:2 个 Agent(Researcher + Writer)
3. Researcher 用 `send` 给 Writer 发消息
4. 前端订阅 `WorkspaceUIBus` ,看到两个 Agent 的对话
5. 加一个 `create_group` ,3 个 Agent 群聊

验证标准:

- 能解释为什么进程内用 Queue、跨进程用 Redis pub/sub
- 2 个 Agent 能互相发消息
- 前端能看到对话流

Codex 怎么用:

- "帮我基于 agplatform swarm 写一个 2 Agent 协作 demo"
- "为什么用 Redis pub/sub 而不是直接 HTTP 通知?"

时间投入:15-18 小时

W11:学 A2A + Orchestrator

为什么学这个:理解 Agent 怎么被发现、怎么路由,建立系统视野。

读什么:

1. agplatform `orchestrator/internal/a2a/manager.go` — A2A Manager
2. agplatform `orchestrator/internal/a2a/resolver/` — Agent 发现
3. agplatform `orchestrator/internal/canary/types.go` — 灰度
4. agplatform `orchestrator/internal/sandbox/types.go` — 沙箱

怎么练(3 步,Go 不需要会写):

1. 在纸上画 Resolver 装饰器链: `Nacos` → `Hybrid` → `Cached`
2. 理解三种路由(direct / rocketmq / hybrid)的取舍
3. 理解灰度的 4 种策略(百分比 / 租户 / 用户属性 / sticky session)

验证标准:

- 能说清楚 Agent Card 是什么
- 能解释 Hybrid Resolver 为什么是"装饰器模式"
- 能说清楚灰度发布解决什么问题

Codex 怎么用:

- "这段 Go 代码帮我解释"
- "装饰器模式在 Resolver 链里怎么体现?"

时间投入:8-10 小时(只读,不写)

W12:综合项目 — 用 Codex 全栈做一个 Agent 应用

为什么学这个:把前 11 周串起来,证明你能独立交付。

项目选择(三选一):

项目 A:个人知识助手

- 前端:React + 你 W2 的 `<AgentChat />`
- 后端:FastAPI + 你 W6 的 LLM 适配器
- 知识:你 W8 的 RAG
- 用户上传 PDF → 切片 → 提问 → Agent 回答

项目 B:多 Agent 写作助手

- 基于你 W10 的 Swarm
- 3 个 Agent:Researcher(查资料)+ Writer(写初稿)+ Reviewer(改稿)
- 前端展示协作过程

项目 C:浏览器自动化

- 基于你 W4 的 Page Agent
- 用户说"帮我登录 XX 网站" → Agent 操作

怎么做(分 4 天):

1. Day 1-2:让 Codex 生成骨架,你审代码、改 API
2. Day 3:接通前后端,跑通主流程
3. Day 4:打磨 UI、修 bug、写 README

验证标准:

- 端到端跑通
- 能给同事 demo
- 代码你能讲清楚每段在干什么

Codex 怎么用:

- 分阶段生成,不要一次生成全部
- "先生成后端骨架,前端我之后写"
- 出 bug 先让 Codex 解释,再让它改

时间投入:20-25 小时

每日习惯(贯穿 90 天)

习惯	时长	方法
读源码	30 分钟	每天专注一个文件,不贪多
写代码	30-60 分钟	用 Codex,但每次都要理解它写的是什么
记笔记	10 分钟	用 Obsidian 或飞书,记录"今天学了什么"
周复盘	30 分钟	每周末回顾,调整下周计划

学习节奏建议

- 不要追求完美:W1 的产出不完美没关系,关键是动起来
 - 用 Codex 当老师:不懂就问,让它解释,不是让它替你学
 - 优先读,次要写:前期多读项目源码,后期再多写
 - 建立里程碑:每完成一周,奖励自己(看个电影、吃顿好的)
-

15. 你现在就能接手的 Codex 全栈任务

15.1 低难度(前端为主)

1. Agent 聊天组件库:把 autobots SSE 抽成通用组件,支持工具卡片、思考过程、错误重试
2. Page Agent Demo:用 page-agent-sdk 做一个"自动填表单"demo
3. 语音 Agent Demo:整合 ASR + LLM + TTS,做一个语音问答

15.2 中难度(前后端结合)

1. MCP 工具市场前端:展示可用 MCP 工具,支持一键启用
2. Agent 编排可视化:用 React Flow 画 Agent 协作图
3. RAG 知识库管理:上传文档、查看切片、测试检索

15.3 高难度(全栈挑战)

1. 完整 Agent 应用:用户提问 → RAG 检索 → LLM 回答 → 工具调用 → 多轮对话
2. 多 Agent 协作场景:Researcher + Writer + Reviewer 协作写文章
3. 浏览器自动化 Agent:用 Page Agent SDK 做网页操作自动化

15.4 用 Codex 的技巧(针对你)

- 先写接口定义(TS interface / Python type),让 Codex 按接口实现
 - 分阶段生成:先生成骨架,再填充逻辑,避免一次性生成太多
 - 用项目已有代码做参考:跟 Codex 说"参考 `useSSEChat.ts` 的写法"
 - 后端不懂的地方:让 Codex 先解释,再让它改
-

16. 关键设计模式速查表

模式	项目位置	前端类比
工厂方法	<code>llm_loader.py</code> 的 <code>loadmodel_*</code>	工厂函数创建组件
策略模式	<code>BaseModelAdapter</code>	不同渲染策略
模板方法	<code>BaseModelAdapter.format_messages</code>	HOC
装饰器	Hybrid Resolver(Nacos→Hybrid→Cached)	HOC 嵌套
观察者	<code>WorkspaceUIBus</code> Redis pub/sub	EventTarget
事件总线	<code>AgentEventBus</code> <code>asyncio.Queue</code>	EventEmitter
权限沙箱	<code>SkillPermissions</code> 最小权限	RBAC
背压	SSE 队列溢出丢弃	throttle/debounce
适配器	<code>OpenAiCompat</code> 一适配多	adapter pattern

17. 名词速查

术语	解释
LLM	大语言模型(GPT/Claude/通义)
Agent	能自主调用工具、循环推理的 AI
Tool / Function Calling	LLM 调用外部函数的协议
MCP	Model Context Protocol,Agent 调工具的标准协议
RAG	Retrieval-Augmented Generation,检索增强生成
Embedding	把文本转成向量(用于相似度计算)
Vector Store	存向量的数据库(Milvus / LanceDB)
Hybrid Search	稠密 + 稀疏混合检索
RRF	Reciprocal Rank Fusion,排名融合算法
Reranker	精排模型(粗排后二次排序)
ASR	语音转文字
TTS	文字转语音
VAD	语音活动检测(检测说话开始/结束)
SSE	Server-Sent Events,服务端流式推送
A2A	Agent-to-Agent,Agent 间通信协议
Swarm	多 Agent 群聊协作
Agent Card	Agent 的"名片"(描述能力)
Sandbox	沙箱隔离(执行代码用)
Canary	灰度发布
Streaming	流式输出(逐字显示)
Tool Use	LLM 输出"要调工具"的信号
Tool Result	工具执行结果,回灌给 LLM
Loop	Agent 的"思考-调工具-再思考"循环
Working Memory	工作记忆(当前对话)
Long-term Memory	长期记忆(跨会话)
Lossless Restatement	无损重述(原始记忆)
Page Agent	浏览器内 Agent
HMAC	哈希消息认证码(Page Agent 握手用)

结语

你已经做了 autobots 前端,SSE / 语音 / Page Agent 这些 Agent 链路里最难的前端部分你都碰过。

你现在要做的不是"转行",而是:

1. **把前端能力深化**(通用 Agent 组件库)
2. **向后端延伸**(用 Codex 帮你写,你读懂+能改)
3. **建立系统视野**(理解 Agent 四大支柱 + 多 Agent 协作)

90 天后,你会成为"能独立用 Codex 全栈交付 Agent 应用"的工程师,这是 2026 年最稀缺的能力。

"Agent 时代的前端,不是画 UI,而是设计人与 AI 协作的交互。"

本文档基于 autobots & agplateform 项目源码整理 生成时间:2026-08-20